G6_NC_OG_09-08

Mar - Teo - Componentes de un Proyecto de desarrollo de SW

> **Clase del martes 15: Scrum**
>
> Es crucial llegar a la clase habiendo leído la **Guía de Scrum 2020** y la **Guía de Expansión 2026** (leen ambas, la expansión es lo nuevo sobre la base de la 2020). Si no se lee, se perderá la clase en lo básico.

- Qué implica planear un proyecto en la forma tradicional?

El proceso del SW

- Transformar ideas, requerimientos y tiempo en un producto

Las 4P

El ciclo de vida del producto tiene dentro ciclos de vida de diferentes proyectos.
Un proyecto está formado por características de producto.
Proyecto es el medio por el cual obtenés un producto.
Los saberes de producto y proyecto son diferentes. Es diferente definir un proyecto a definir un producto.
Las 4P son las dimensiones por las que se miden las disciplinas de sw.
Las personas son el elemento más importante que tenemos. Se suman a los proyectos pero son un elemento clave.

Proyecto: unidad de gestión del trabajo que produce un resultado único y distinto cada vez.

Proyecto, producto y proceso son cosas distintas, son saberes distintos y roles distintos.

- **Roles de producto**: analistas, desarrolladores, arquitectos, líderes técnicos, testing, análisis funcional, diseño de experiencia de usuario (UX/UI).
- **Esta materia hace énfasis en proyecto**: líderes de proyecto, gestión, coordinación. Cubrimos testing y despliegue pero no es el foco principal.

Procesos definidos y procesos empíricos

La diferencia entre los dos tipos de proceso se basa en la **actitud frente a los cambios** y en **qué base parte** cada uno.

> ⚠ **Concepto clave**
>
> No es razonable decir en ningún ámbito que existe un contexto donde no se puedan hacer cambios. Si un cliente te dice "esto no es lo que yo necesitás", no podés responder "pero yo estoy usando un proceso definido, los cambios no van". Eso no se sostiene ni cinco minutos en la vida real. Lo que varía es la **actitud** con la que cada tipo de proceso enfrenta los cambios.

- **Definidos**: creen que se pueden establecer de antemano, detalladamente, qué cosas hay que hacer para construir software (actividades, entradas, salidas, métricas, roles). La experiencia puede extrapolarse a otros proyectos. Se asume que si hacés lo que dice el proceso, vas a lograr ser repetible y predecible (estimar bien basado en datos históricos). Usan técnicas de estimación que sistematizan información de proyectos pasados como datos de referencia.
- **Empíricos**: en escenarios de incertidumbre, actitud rápida para ejecutar cambios. Generan algo para que el cliente lo vea y tenga feedback rápido (máximo un mes por iteración). **La experiencia no se extrapola a otros proyectos.** Cada proyecto es diferente porque las personas, el cliente y el producto son diferentes. La experiencia se genera como equipo a través de iteraciones frecuentes de software funcional. Se toma la experiencia de las iteraciones anteriores **dentro del mismo proyecto**, pero no se copia la de otros proyectos. Experiencias colectivas y a lo que te podés comprometer como equipo.

> ⓘ **Instanciación y adaptación del proceso**
>
> El proceso es una definición **teórica** de lo que tenés que hacer. Pero cada vez que lo vas a ejecutar en el contexto de un proyecto, tenés que determinar si lo que la organización definió te sirve o no, y cuánto de eso te sirve. No es que "hoy me levanté y me pintó que no voy a documentar la arquitectura": si el proceso dice que hay que documentarla, te vas a justificar muy bien por qué en tu caso no aplica.

> 🔥 **Consejo del profe**
>
> No digan que un proceso definido es "misma entrada → misma salida" porque no es así. Siempre es mejor tener más información que menos. Los empíricos dicen que no van a

tomar como referencia la experiencia de otros, sino generar la propia. Pero no se les borra la memoria a la gente — la experiencia individual siempre queda.

El mundo del software es un mundo complejo y ambiguo. Esta materia no tiene recetas: tiene enfoques donde uno piensa una cosa y lo defiende y el otro piensa otra y lo defiende. Ninguno es 100% verdad ni 100% mentira. Todos tienen cosas rescatables y cosas que hacen agua. Y ni hablar cuando eso lo querés llevar a la realidad.

Diferencia entre proceso y ciclo de vida

Pregunta para el parcial

Diferencia entre proceso y ciclo de vida del proyecto.

- **Proceso**: define las cosas que hay que hacer para hacer software. Define actividades, entradas, salidas, roles, métricas. Es la definición teórica.
- **Ciclo de vida del proyecto**: determina **cuánto** de cada una de esas actividades del proceso vas a hacer cada vez, los puntos de control, cuándo pasás de una etapa a otra. **Define CÓMO usar el proceso.**

El ciclo de vida es el que te dice cómo usar el proceso. El proceso unificado recomienda un ciclo de vida iterativo incremental, pero muchísima gente usa el proceso unificado con un ciclo de vida en cascada y por eso fracasa.

Tipos de ciclo de vida

Los tipos genéricos son:

- **Iterativo**
- **Secuencial**
- **Recursivo**

De ahí se deducen ciclos de vida específicos:

- Dentro de los secuenciales: **cascada**
- Dentro de los iterativos: **iterativo incremental** (el que usa el Proceso Unificado)

Para procesos empíricos: solo iterativo

Para los procesos empíricos **solo se usa iterativo**. ¿Por qué? Porque busca aprender en base a las experiencias propias del equipo. La urgencia de obtener conocimiento hace que los ciclos sean lo más pequeños posible (iteraciones frecuentes) para generar rápidamente conocimiento y ajustar lo que haya que ajustar.

> **Referencia**
>
> Ver *Rapid Development*, capítulo 7.

Qué es un proyecto

> **Pregunta para el parcial**
>
> Características de un proyecto.

Un proyecto es una **unidad de gestión del trabajo** que tiene las siguientes características:

- **Resultado único y diferente cada vez**: si el resultado no es único, no es un proyecto. Por eso se usan proyectos para gestionar software — el software no se puede fabricar en líneas de producción masivas como un producto tangible.
- **Duración limitada**: tiene fecha de inicio y fecha de fin. Aunque no es garantía de que se cumplan las fechas. La responsabilidad del PM es poder cumplir los deadlines. En algún momento el producto se tiene que terminar. A diferencia de una línea de producción serial, que no tiene fecha fin.
- **Orientado a objetivos**: tiene que ser **claro** y **alcanzable**.
  - **Claro**: todos entendemos lo mismo respecto a lo que queremos lograr como resultado del proyecto.
  - **Alcanzable**: haciendo lo mejor que podemos todos, eso se puede lograr.
  - Un objetivo puede ser claro y **no** alcanzable (ejemplo: "hacer una billetera virtual para pagos en una semana" — está claro qué hay que hacer, pero no hay tiempo para lograrlo).
  - Un objetivo **no** puede ser alcanzable y no claro (¿cómo sabés que algo es alcanzable si ni siquiera tenés claro qué tenés que hacer?).

> **Tip del profe: Ejemplos**
> Un ejemplo es algo concreto, una instancia específica donde demostrás o aclarás un concepto. Cuando se pida ejemplos en parciales o finales, dar una **situación**

concreta. Ejemplo: "queremos hacer una landing page" no es claro — ¿qué información quiero que muestre? "Necesitamos un software para una billetera virtual pero solo tenemos una semana" — eso sí es un ejemplo claro y concreto.

- **Tareas interrelacionadas**: es la manera en la cual manejamos la complejidad. Dividimos el proyecto en tareas que sean manejables, que tengan objetivos parciales, pero que todos esos objetivos parciales confluyan en el objetivo final del proyecto. Por eso las tareas tienen que estar interrelacionadas.

La importancia de la planificación

> **"Lo importante no es el plan, sino planificar"**
>
> La accción de planificar te hace analizar información y aprender. El plan es el artefacto resultante, pero lo valioso es lo que aprendés mientras estás planificando.

Proceso resultante de la planificación de un proyecto: lo importante es el **análisis de información**. Si planificás solo para llenar un template porque alguien te lo pide, deja de ser valioso.

No hay bala de plata (Silver Bullet)

No hay ninguna solución tecnológica que por sí sola prometa una mejora de 10x en productividad, fiabilidad o simplicidad.

Esencia vs accidente del software (Fred Brooks)

- **Esencia**: lo inherente a la construcción del software — la complejidad conceptual, la especificación, el diseño y prueba de los constructos conceptuales. No es el trabajo manual de escribir código.
- **Accidente**: lo "tangible" del desarrollo — el código, las herramientas, los frameworks, etc.

La gran mayoría de los avances en desarrollo de software han apuntado al **accidente** y no a la **esencia**, que es donde están las raíces y los grandes problemas. El problema es que creemos que las soluciones que inventamos van a atacar la esencia y terminan resolviendo el problema equivocado.

> ⚠ **La bala de plata moderna**
>
> Cuando apareció la POO parecía la panacea. Hoy pasa lo mismo con la **inteligencia artificial generativa**: hace avances exponenciales pero sobre el terreno **accidental**, no sobre la esencia. La IA generativa tampoco está atacando los problemas esenciales del desarrollo.

> ⓘ **Referencia**
>
> *No Silver Bullet — Essence and Accident in Software Engineering*, Fred Brooks, 1986.

El hombre mítico (Brooks)

> ⓘ **Referencia**
>
> *The Mythical Man-Month*, Fred Brooks, 1976.

El costo más representativo de hacer software es el costo de las **personas** que intervienen en el proceso de construcción. Por eso el libro se llama "hombre-mes": históricamente se habla de esfuerzo en horas-hombre (aunque el término tiene un problema de género, se deja la referencia original).

Restricciones secuenciales

"Gestar un hijo toma 9 meses sin importar cuántas mujeres asignes." Hay situaciones donde no se admite asignación de recursos — un bebé lo hace una mujer en 9 meses, 9 mujeres no hacen un bebé en un mes.

Agregar personal a un proyecto atrasado lo retrasa más

Inyectar nuevos ingenieros a un ecosistema tardío genera una cascada de fricción operativa. Nadie trabaja: ni lo nuevo (hay que formarlos), ni lo viejo (dedican tiempo a enseñar).

La siguiente solución habitual es horas extras o fines de semana. Funciona por un periodo muy corto, pero el trabajo intelectual requiere descanso: si la persona no duerme bien, se incrementan exponencialmente los errores. El remedio sale más caro que la enfermedad.

La integridad conceptual es la meta suprema

> **"Organismo con dos cabezas se muere"**
>
> — Profe de estructura de las organizaciones

La analogía con un cirujano: hay mucha gente trabajando en la intervención, pero **quien toma las decisiones es uno solo**. Las decisiones vinculadas a la calidad del software que vamos a entregar deberían descansar sobre **una sola cabeza**. Alguien tiene que asumir la responsabilidad de liderazgo y coordinar hacia dónde va el producto.

El sistema más peligroso es el segundo que construís

Tenemos una tendencia a agregarle complejidad y características al pedo. El cliente pide, nosotros ofrecemos, y el otro dice "bueno, sí". Hay que aplicar el principio **YAGNI** (*You Ain't Gonna Need It*): no lo vas a necesitar.

> ⚠ **Dato mundial**
>
> El **45% de las características** incluidas en un producto de software **no se usan jamás**. Aplicando YAGNI podríamos reducir esto y tener una versión del producto lo antes posible.

> ⓘ **Referencia**
>
> Ver slides del tema de gestión de productos, una de las primeras gráficas con estadísticas de software a nivel mundial.

Calidad del producto de software

Un producto de software tiene calidad cuando **satisface las necesidades y expectativas** del cliente (integridad de producto, como vimos en gestión de configuración). La calidad **nunca debería sacrificarse**. Se negocian compromisos, no calidad.

Las decisiones que se toman en el ámbito del proyecto se ven reflejadas en el producto final. Si te comprometés a entregar un producto con la mitad del tiempo necesario, se va a sacrificar

la calidad.

Documentación del producto

Las decisiones importantes que se van tomando respecto al producto tienen que haber **escritas**: qué se hizo, por qué, cómo se resolvieron los requerimientos.

> **La escritura requiere un nivel de precisión distinto que el "biri-biri"**
>
> El hecho de escribir hace que nos demos cuenta de las cosas que no sabemos, que no tenemos claras, que tenemos que indagar. Por eso es importante que exista documentación del producto. No importa el "cómo" (pizarrón, afiche, repo), pero importa el "qué".

No delegar el pensamiento a herramientas de IA

> **Consejo de vida**
>
> No pueden delegarle la acción de pensar a nadie, a ninguna herramienta, a ninguna persona. Mientras menos piensen, menos capacidad de pensar van a tener. Usen la IA para facilitarles la vida, pero **piensen ustedes**. Tengan pensamiento crítico.

Si la IA les escribe user stories, lo mínimo que deberían hacer antes de entregarlas es **corroborarlas**. No entregar algo sin verificar si está bien o mal.

> **Ejemplo real**
>
> Alguien le pidió a ChatGPT que le escriba un mail para preguntar algo a la profesora. Se olvidó de quitar la parte donde decía "olvidate de esta casa aparte". La profe recibió el mail completo. También pasó que mails escritos por IA le decían "estimado profesor" (a una profesora), y no los revisaban antes de mandar.

Qué es la administración de proyecto

La **administración de proyecto** es la disciplina que se encarga de la planificación y el seguimiento de los proyectos para que estos cumplan con sus objetivos. Tiene dos disciplinas principales:

1. **Planificar**: definir cómo vamos a alcanzar los objetivos.
2. **Monitorear y controlar**: comparar el avance real contra lo planificado.

> ⚠ **Relación entre planificar y monitorear**
>
> - Se **puede** hacer un plan y no monitorear — pero está mal. Ese plan se transforma en desperdicio.
> - **No se puede** hacer monitoreo sin un plan previo. El monitoreo es una actividad de comparación. ¿Con qué comparás el avance si no tenés un estándar (el plan) contra qué compararlo?

La restricción triple

El proyecto se mueve sobre **tres dimensiones**:

Dimensión	Qué significa
Alcance	Los requerimientos, lo que el cliente necesita
Tiempo	Cuándo tiene que estar listo
Costo	Los recursos y su costo derivado

> 🖉 **Sobre recursos**
>
> Cuando hablamos de recursos no hablamos técnicamente de personas, sino del **trabajo de las personas**. El costo más significativo de hacer software es el trabajo humano.

La gestión de proyectos intenta lograr un **equilibrio** entre estas dimensiones para cumplir los objetivos **sin afectar la calidad**. La calidad es la variable que se suele afectar primero cuando el equilibrio se desajusta.

¿Quién controla qué?

La **única variable** que debería controlar el cliente es el **alcance** (qué quiere y necesita). Él es quien sabe lo que necesita. Los recursos y el tiempo se derivan de los requerimientos.

En la práctica

Idealmente, como líder de proyecto, deberías tener control sobre **2 de las 3 variables**. Si el cliente te condiciona las tres (qué quiere, cuándo lo quiere y cuánto paga), no tenés margen de acción para tomar decisiones como líder. Si te dicen "quiero esto en este tiempo con este presupuesto"... casi que te están diciendo "hacelo vos solo".

Negociación

Lo más factible para negociar es en términos de **alcance**, porque:

- El 100% de los requerimientos al principio es muy difícil de determinar
- El 45% de las features que la gente pide después no se usa
- Esa es tu base de negociación

Si no se puede negociar alcance, se negocia tiempo (pero recordar: más gente en un punto estorba, y el trabajo de las personas es el costo más significativo). Si se puede, se negocia presupuesto.

Gestionar compromisos

Se negocian **compromisos**, no estimaciones. Las estimaciones son técnicas, los compromisos son lo que te atás a entregar.

La gestión tradicional y la resistencia al cambio

La gestión tradicional parte de la base de que el cliente fija el alcance y nosotros, comprendiendo esos requerimientos, determinamos recursos y tiempo (se derivan del alcance). Esa es la razón por la cual **se resiste a cambiar**: cuando el cliente viene con cambios avanzado el proyecto, ya se firmó un contrato con alcance, tiempo y costo definidos.

En agilismo, el product owner es parte del equipo. Técnicos y no técnicos trabajan juntos.

El líder de proyecto y los equipos

El **líder de proyecto** es el que:

- **Estima** el proyecto
- **Arma el plan**

- **Toma todas las decisiones** relacionadas al proyecto (siempre considerando las tres dimensiones)
- **Coordina** al equipo de proyecto
- **Establece el vínculo** con los otros involucrados (stakeholders): clientes, project manager (jefe del líder), gerentes funcionales de la empresa. Es la interfaz entre el equipo y el resto.

Equipo de proyecto tradicional

En gestión tradicional, el líder tiene a cargo un equipo conformado por roles especializados y distribuidos funcionalmente:

- Programadores
- Analistas
- Líderes técnicos / arquitectos
- Gente que hace testing
- Gente que hace calidad
- Gente que hace liberación
- Diseño de experiencia de usuario (UX/UI)

Cada persona tiene su función específica, como posiciones en el fútbol: el arquero es arquero, el delantero es delantero. Si ponés al arquero a jugar de otra cosa, probablemente no sea tan bueno.

Equipo ágil: multidisciplinar

En agilismo, los equipos apuntan a ser **multidisciplinares**: cada uno debería estar en condiciones de "jugar en toda la cancha". No solo programar, sino también negociar requerimientos con el cliente, hacer prototipos, documentar, hacer manual de usuario, etc.

Un full stack en agilismo sería alguien que puede hacer front, back, base de datos, **y además** comprometerse con una user story, empezarla y terminarla (incluyendo toda la documentación que acompaña).

> ⚠ **Diferencia clave con lo ágil**
>
> Lo que **no es negociable** en el agilismo:
>
> - La **autogestión** — no hay un jefe que te diga qué hacer
> - **Estima el que hace el trabajo** — no el líder
> - El **compromiso** de cada uno — cada uno se asume un compromiso y asume el compromiso de cumplir

En gestión tradicional: hay un jefe que asigna anticipadamente, él estima y decide quién hace qué.

Equipos funcionales vs multidisciplinares

Desde el punto de vista teórico, en gestión tradicional los equipos son **funcionales** (cada uno con su rol especializado). En agilismo son **multidisciplinares**. En la práctica, muchas empresas que funcionan con gestión tradicional tienen dificultad para hacer esta transición porque la gente fue contratada para roles específicos (analista funcional, tester, etc.) y no necesariamente puede "jugar en toda la cancha".

Algunas prácticas se pueden mezclar (híbridos), pero solo si son **compatibles con la filosofía** de cada enfoque. Lo que **no** se puede mezclar: ciclos de vida secuenciales con un proceso empírico.

Composición del plan de proyecto

Pregunta para el parcial

¿Qué aspectos debe responder un plan de proyecto?

Un plan de proyecto necesita respuestas a estos aspectos:

1. **Objetivo del proyecto**
2. **Alcance del proyecto**
3. **Proceso** que vas a usar
4. **Ciclo de vida**
5. **Estimaciones** asociadas al proyecto
6. **Riesgos**
7. **Asignación de recursos**
8. **Cronograma** (programación del proyecto)

El cronograma NO es todo el plan

Mucha gente cree que hacer un diagrama de Gantt es planificar el proyecto. El cronograma es **una partecita** de la planificación, y normalmente es la **parte final** — se

hace cuando ya resolviste todas las demás preguntas. No es casualidad que esté ubicado al final: es el derivado de las respuestas a todos los demás aspectos.

Alcance del proyecto vs alcance del producto

Pregunta para el parcial

Diferencia entre alcance del proyecto y alcance del producto.

	Alcance del producto	**Alcance del proyecto**
Qué define	Todas las cosas que el producto va a **hacer**	Todo el trabajo (y solo el trabajo) que hay que hacer para cumplir con el objetivo
Dónde se escribe	En la **ERS** (Especificación de Requerimientos / documento de requerimientos)	En el **plan de proyecto**
Ejemplo	"Quiero un software que gestione mi agenda, que mande alarmas, que funcione desde el celular"	Analizar requerimientos, definir tecnologías, diseñar arquitectura, desarrollar, testear, etc.

Ejemplo concreto del profe

"Quiero un producto que gestione mi agenda con cosas de trabajo y personales, que mande alarmas, que funcione desde el celular." → Eso es el **alcance del producto** (lo que el producto tiene que hacer).

¿Qué tendrían que hacer ustedes para entregarme ese producto? Analizar requerimientos, definir tecnologías, diseñar arquitectura, desarrollar, etc. → Eso es el **alcance del proyecto** (todo el trabajo que hay que hacer).

El alcance del proyecto se deriva del proceso: ¿qué tengo que hacer para entregarle el producto a esta persona? Te vas a fijar al proceso, donde dice qué hay que hacer para hacer software.

Estimación en gestión tradicional

La estimación tradicional tiene otro foco: apunta a la **precisión y certeza** (qué día, qué hora, cuánto va a salir). Es más costosa y requiere más esfuerzo que la estimación ágil. Además, apunta a **estimaciones absolutas** (nosotros somos mejores haciendo estimaciones **relativas**, como las de ágil).

Orden de la estimación tradicional

El proceso de estimación sigue este orden:

1. **Tamaño** → ¿cuán grande es lo que hay que hacer?
   - Se estima el **alcance del producto** (independiente de quién lo va a hacer y de cuánta plata tiene el cliente)
   - En gestión tradicional se cuentan **casos de uso** (lo más cercano a tener requerimientos al inicio del proyecto)
   - Se clasifican: cuántos casos de uso simples, cuántos complejos, etc.
   - > **✎ ¿Por qué casos de uso y no líneas de código?**
     > Las estimaciones son tempranas — rápidamente tenés que dar una estimación y lo que tenés al principio del proyecto son requerimientos, no sabés cuántas líneas de código va a tener.
2. **Esfuerzo** → derivás el esfuerzo a partir del tamaño
   - **Horas persona lineales**: es como si trabajara una sola persona haciendo una cosa a la vez, después de la otra
   - El resultado es una **bolsa de horas**: 1000 horas, 10.000 horas, 20.000 horas
   - > **⚠ Error común**
     > Si el líder de proyecto estima el esfuerzo sin consultar al equipo, puede ser una fuente de errores. Él estima con su conocimiento, pensando que es él quien va a hacer el trabajo, o consulta a un experto que no es el que realmente va a trabajar.
3. **Calendario** → transformás la bolsa de horas en cuándo
   - Tenés que considerar:
     - Cuánta gente va a trabajar
     - Qué trabajo se puede hacer en paralelo (solapadamente)
     - Cuántas horas por día trabaja la gente
     - Cuántos días a la semana
     - Disponibilidad de la gente (¿exclusivo o parcial?)
4. **Costo** → lo último que se calcula

- Es simplemente un cálculo derivado de lo anterior
- Sabés cuál es el costo de cada persona y lo multiplicás por las horas que le va a dedicar
- El costo del esfuerzo lleva más o menos un **80% del costo total del proyecto**
- El otro 20% son costos indirectos y cosas depreciables

> **✎ El costo de la IA generativa como nuevo ruido**
> Al igual que en su momento hizo ruido la conectividad (internet), ahora está haciendo ruido el **costo de la inteligencia artificial**. Empresas que habían adoptado IA empezaron a recibir facturas enormes por consumo de tokens y dijeron "bueno, a ver, no sé si esto está tan bueno". Las primeras métricas serias sobre la incidencia del costo de la IA en la construcción de software están empezando a salir.

> **🔥 Resumen del orden**
>
> **Tamaño → Esfuerzo → Calendario → Costo**
> Cada paso se deriva del anterior. El costo es el último porque es simplemente una multiplicación de lo que ya calculaste.

Recursos críticos

Siempre vas a disponer de **menos de lo que necesitás**. Hay recursos que son críticos y hay que administrarlos con cuidado:

- **Tokens de IA**: algo que no administrás vos pero que necesitás controlar (ejemplo: el costo de la IA generativa)
- **Ancho de banda de internet**: puede ser un recurso crítico
- **Licencias de software**: si tenés pagadas 3 licencias concurrentes y hay 10 personas en el equipo, tenés que administrar cómo las distribuís
- **Sensores, hardware especializado**: si estás haciendo un sistema de alarma contra incendios y el cliente te dio un solo sensor para probar, tenés que ver cómo te manejás

> **✎ Recurso crítico vs recurso normal**
>
> Una **computadora** no es un recurso crítico (se supone que cada uno tiene una). Pero si resulta que en un proyecto no tiene cada persona una computadora, replanteate si lo van a poder hacer. Los recursos críticos condicionan el costo, sí, pero lo mejor es siempre que no sean críticos. La mayoría de las empresas no los miden ni los consideran como costo

fijo. Si vas a decidir considerar algo como costo fijo, debería ir al final — porque todo lo demás que vos decidas va a impactar en el costo.

Riesgos en proyectos

Pregunta para el parcial

Diferencia entre riesgo y problema.

Riesgo ≠ problema

Un **riesgo** tiene asociado un factor de **probabilidad**: puede pasar o no pasar. Un **problema** es certero — ya sabés que te va a pasar.

⚠ Error común

Se confunde riesgo con problema. Decir "si no nos alcanza el tiempo" **no es un riesgo** — es un problema, porque ya sabés que no te alcanza. No le asignés probabilidad porque es certero. Si le asignás probabilidad 1, ya no es riesgo, es problema, y la **acción que tomás es distinta**.

Una definición de riesgo: *un problema esperando suceder*. Otra: *probabilidad de ocurrencia de un impacto negativo en el proyecto*.

Exposición del riesgo

El riesgo se mide con la **exposición**, que es el producto de dos dimensiones:

$$\text{Exposición} = \text{Probabilidad de ocurrencia} \times \text{Impacto}$$

- **Probabilidad**: un número entre 0.01 y 0.99 (si le asignás 1 ya no es riesgo)
- **Impacto**: un número entre 0.01 y 1 (podría ser 1 si es destrucción total, pero si es 1 no multiplicás nada)

La exposición **homogeneiza** los riesgos para poder compararlos:

- Un riesgo con probabilidad muy baja pero impacto muy alto (ej: pongan una bomba en el edificio donde trabajo)
- Otro con probabilidad alta pero impacto menor (ej: se corta el agua)

Ambos pueden tener exposiciones comparables, y con ese número sabés cuál es más grave.

Gestión proactiva de riesgos

En un proyecto sano se espera **gestión proactiva**:

1. **Identificar** los riesgos lo antes posible
2. **Cuantificarlos** (asignar probabilidad e impacto → calcular exposición)
3. **Definir acciones** de mitigación o contingencia para cada uno

Esto se hace dentro del plan de proyecto, en etapas tempranas. Genera conocimiento y experiencia en la organización que prepara para enfrentar mejor la situación a lo largo del proyecto y de la organización.

Riesgos típicos en proyectos de SW

Los riesgos más comunes en el top:

1. **Requerimientos**: cambios, malinterpretación, ambigüedad
2. **Estimaciones**: sobreestimar o subestimar
3. **Personas**: que la gente clave se vaya antes de que el proyecto finalice, o que no se tenga la gente adecuada
4. **Tecnología**: cuanto más nueva y menos probada, más riesgo hay. Es inherente al software — trabajamos con tecnología, y eso suma riesgo adicional

> **Lo más difícil del software (Brooks)**
>
> Lo más difícil de hacer software es decidir **qué** software es el que es (especificar, diseñar). Eso es inherente. Y a eso súmale que trabajamos con tecnología, que se suma riesgo adicional.

Cuando sos consciente de que estás empezando a desarrollar un producto que no tenés idea de qué hace o qué quiere el cliente, eso **no es un riesgo — es un problema**. No tiene probabilidad asociada, es certero.

> **Consejo del profe**
>
> Hagan un esfuerzo por identificar los riesgos lo antes posible y por hacer un análisis cuantitativo: transformarlo en un número para que deje de ser una percepción. Ese número (la exposición) les sirve para comparar cuál es más grave y priorizar.

Nota sobre Scrum (para la próxima clase)

Scrum es un framework que pide equipos de **7 ± 2 personas** (mínimo 5, máximo 9). No es un capricho: tiene que ver con la gestión de la complejidad y el ruido de comunicación. Por qué hay un mínimo de gente en Scrum es una pregunta para la clase que viene.

ⓘ Referencia bibliográfica

Ver referencias sobre ciclos de vida en la bibliografía de la materia.